Scientific Calculator with DevOps Project Report

Roll Number: MT2025715

Name: Manzil Baruah

Links

- **GitHub Repository:** https://github.com/hackingjack139/devops-calc
- **Docker Hub Repository:** https://hub.docker.com/r/hackingjack139/devops-calc

1. Introduction: What and Why of DevOps?

What is DevOps? DevOps is a set of practices, cultural philosophies, and tools that bridges the gap between software development (Dev) and IT operations (Ops). It aims to establish a collaborative environment where building, testing, and releasing software can happen rapidly, frequently, and more reliably.

Why DevOps? Traditional software development often suffers from "silos" where developers write code and toss it over the wall to operations teams to deploy, leading to miscommunication, slow delivery, and unstable environments. DevOps solves this by:

- **Speed & Continuous Delivery:** Automating the pipeline allows for faster releases and immediate feedback loops.
- **Reliability & Quality:** Automated testing and continuous integration ensure that bugs are caught early before reaching production.
- **Scalability & Consistency:** Configuration management and containerization (like Docker) ensure that the application runs the exact same way on a developer's laptop as it does in production.
- **Monitoring & Observability:** Real-time logging (like the ELK stack) allows teams to instantly track application health and user interactions.

2. Tools Used

In this project, I implemented a complete DevOps lifecycle for a Python-based Scientific Calculator using the following tools:

1. **Source Control Management:** GitHub
2. **Testing Framework:** Pytest
3. **Continuous Integration & Delivery (CI/CD):** GitHub Actions
4. **Containerization:** Docker & Docker Hub
5. **Configuration Management & Deployment:** Rundeck
6. **Monitoring & Logging:** ELK Stack (Elasticsearch, Logstash, Kibana) + Python JSON Logger

---

3. Step-by-Step Implementation

Step 1: Source Control Management (GitHub)

Brief: Source Control Management (SCM) is used to track changes to the codebase, collaborate with team members, and trigger automated pipelines when new code is pushed. I used **GitHub** for this project.

Setup & Explanation:

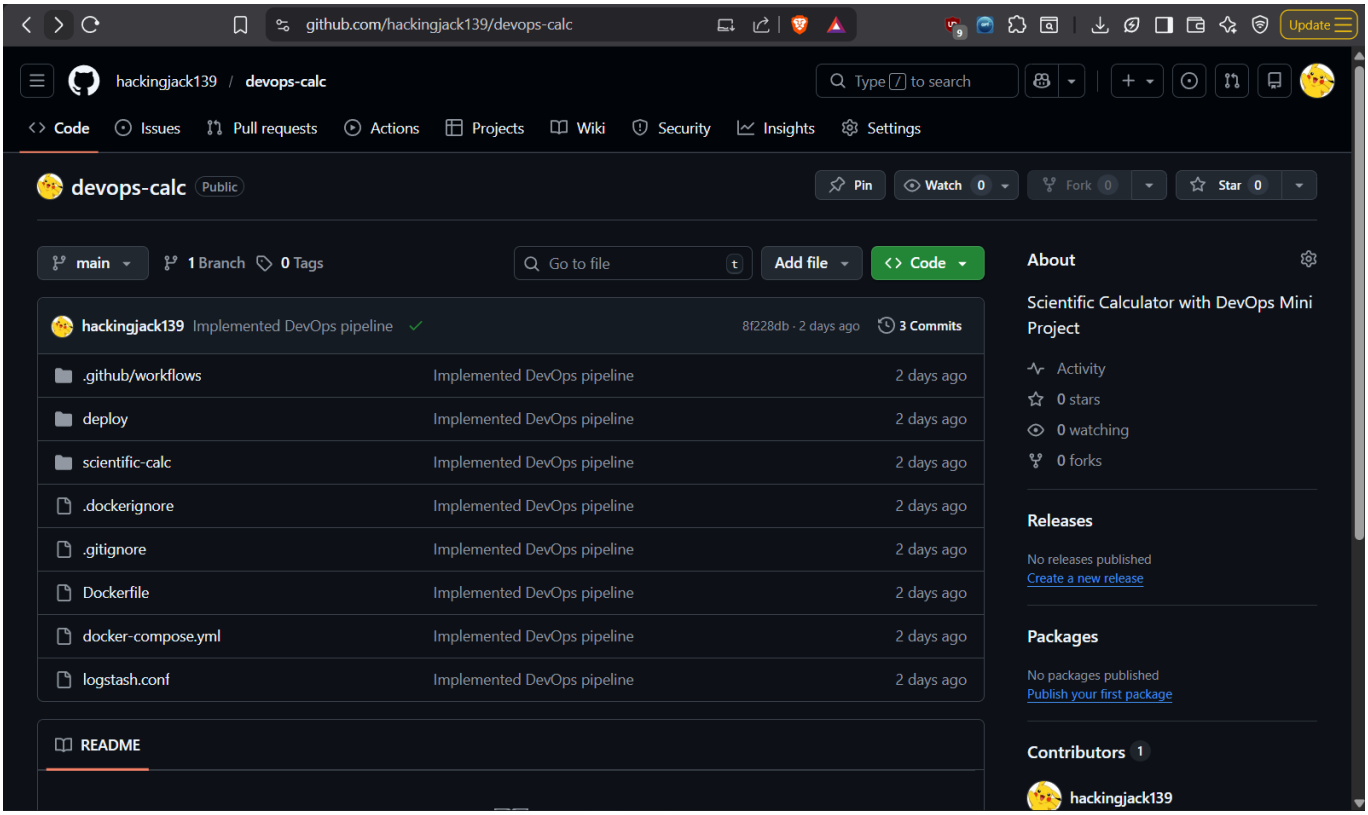
1. Created a new repository on GitHub named `devops-calc`.
2. Initialized the local project folder with Git.
3. Added the GitHub repository as the remote origin.
4. Committed the calculator source code (`app.py`, `test_app.py`) and pushed it to the `main` branch.

Commands Used:

```
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/hackingjack139/devops-calc.git
git push -u origin main
```

Links:

- GitHub Repository: `https://github.com/hackingjack139/devops-calc`



Step 2: Automated Testing (Pytest)

Brief: Automated testing ensures that the application's core logic works as intended before it is ever built or deployed. I used **Pytest** to test the mathematical functions of the calculator.

Setup & Explanation: To enforce code quality, I created a comprehensive automated test suite (`test_app.py`) covering all mathematical operations: square root, factorial, natural logarithm, and power functions. By executing these tests via Pytest, I can confidently verify that the core application logic behaves correctly under various input scenarios before advancing to the build stage.

Commands Used:

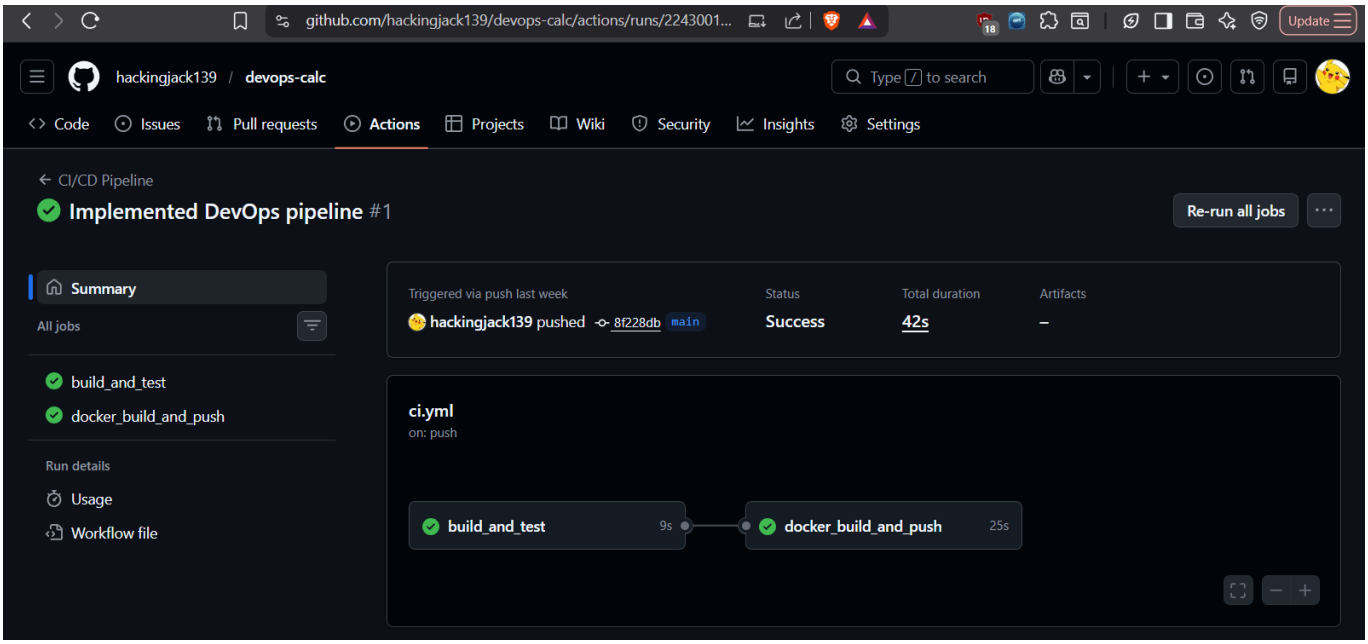
```
python -m pip install pytest
pytest test_app.py -v
```

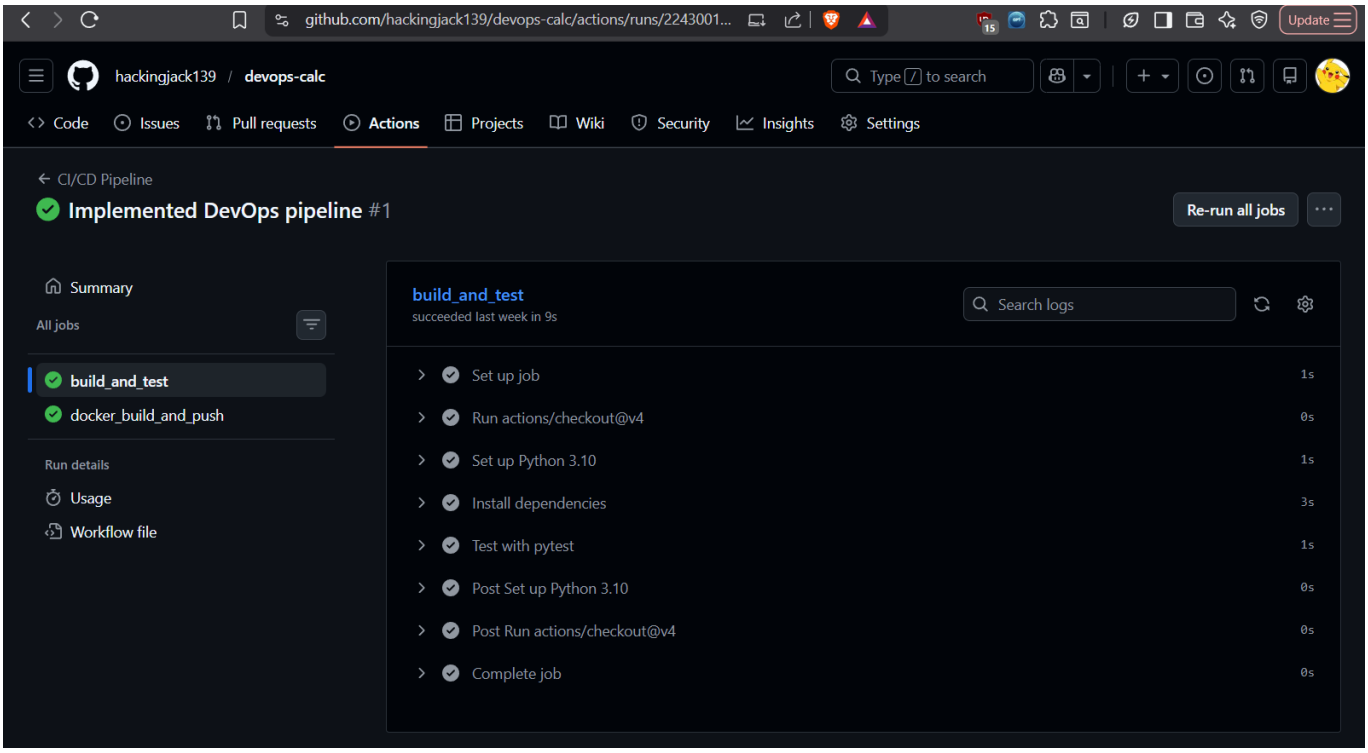
```
~\..\SPE Calc  main  cd scientific-calc
~\..\..\scientific-calc  main  pytest test_app.py -v
============================= test session starts ==============================
platform win32 -- Python 3.13.7, pytest-8.3.4, pluggy-1.6.0 -- C:\Users\manzi\AppData\Local\Programs\Python\Python313\python.e
xe
cachedir: .pytest_cache
rootdir: C:\Users\manzi\Downloads\SPE Calc\scientific-calc
plugins: anyio-4.12.1
collected 16 items

test_app.py::TestSquareRoot::test_square_root_positive PASSED            [  6%]
test_app.py::TestSquareRoot::test_square_root_zero PASSED                [ 12%]
test_app.py::TestSquareRoot::test_square_root_decimal PASSED             [ 18%]
test_app.py::TestSquareRoot::test_square_root_negative PASSED            [ 25%]
test_app.py::TestFactorial::test_factorial_positive PASSED               [ 31%]
test_app.py::TestFactorial::test_factorial_zero PASSED                   [ 37%]
test_app.py::TestPowerFunction::test_power_function_positive_base_exponent PASSED [ 75%]
test_app.py::TestPowerFunction::test_power_function_positive_base_negative_exponent PASSED [ 81%]
test_app.py::TestPowerFunction::test_power_function_zero_base_positive_exponent PASSED [ 87%]
test_app.py::TestPowerFunction::test_power_function_zero_base_zero_exponent PASSED [ 93%]
test_app.py::TestPowerFunction::test_power_function_zero_base_negative_exponent PASSED [100%]

============================== 16 passed in 0.15s ==============================
~\..\..\scientific-calc  main
```

Step 3: Build & Continuous Integration (GitHub Actions)

Brief: Continuous Integration (CI) automatically builds and tests code every time a change is pushed to the repository. I used **GitHub Actions** as my CI server.

Setup & Explanation: I created a workflow file (`.github/workflows/ci.yml`) that triggers on every push to the `main` branch. GitHub spins up an Ubuntu cloud runner, installs Python 3.10, installs my `requirements.txt`, and automatically runs the `pytest` suite. If the tests fail, the pipeline stops, preventing broken code from being deployed.





Step 4: Containerization (Docker)

Brief: Containerization packages the application code along with its dependencies (Python, libraries) into a single, portable unit. I used **Docker**.

Setup & Explanation: I wrote a `Dockerfile` that starts from a lightweight python image (`python:3.10-slim`). It copies my application code, installs dependencies, and sets `python app.py` as the default application execution command. A `.dockerignore` file was also created to prevent local logs and testing caches from bloating the container.

Script (`Dockerfile`):

```
FROM python:3.10-slim
WORKDIR /app
COPY scientific-calc/ .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
```

```
~\..\..\scientific-calc  main  cd ..
~\..\SPE Calc  main  docker build -t devops-calc .
[+] Building 18.5s (9/9) FINISHED                                docker:desktop-linux
 => [internal] load build definition from Dockerfile                                0.1s
 => => transferring dockerfile: 187B                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.10-slim                 3.0s
 => [internal] load .dockerignore                                                   0.1s
 => => transferring context: 90B                                                    0.0s
 => [1/4] FROM docker.io/library/python:3.10-slim@sha256:4b0a8ebf16cf4563f3d37      4.5s
 => => resolve docker.io/library/python:3.10-slim@sha256:4b0a8ebf16cf4563f3d37      0.0s
 => => sha256:7fe6b279891367efd009a54b83403e92c54fe9b9e47c0e5e4685 252B / 252B      0.9s
 => => sha256:aecf3e6b3e04ba467f457952ea77eb6cebfb10ea3231a822 1.29MB / 1.29MB      0.7s
 => => sha256:44df7c9b01189cee78fe34391e9a58e4a1d449a932b411 13.82MB / 13.82MB      1.8s
 => => extracting sha256:aecf3e6b3e04ba467f457952ea77eb6cebfb10ea3231a822d131c      0.7s
 => => extracting sha256:44df7c9b01189cee78fe34391e9a58e4a1d449a932b41113d54fa      2.4s
 => => extracting sha256:7fe6b279891367efd009a54b83403e92c54fe9b9e47c0e5e46853      0.0s
 => [internal] load build context                                                   0.1s
 => => transferring context: 77.47kB                                                0.1s
 => [2/4] WORKDIR /app                                                              0.2s
 => [3/4] COPY scientific-calc/ .                                                   0.2s
 => [4/4] RUN pip install --no-cache-dir -r requirements.txt                        7.6s
 => exporting to image                                                              2.5s
 => => exporting layers                                                             1.3s
 => => exporting manifest sha256:b50bb3cd18bd029b9d6fbbc5efd284fe283f9644fe683      0.0s
 => => exporting config sha256:7c03d56e2f49f56ade498c1c67b316387242b63593c551c      0.0s
 => => exporting attestation manifest sha256:c964d92dad73f8e26f32c98562f671e48      0.0s
 => => exporting manifest list sha256:8a43da299317ae2b9c41ef2639263816d4f4c5b1      0.0s
 => => naming to docker.io/library/devops-calc:latest                               0.0s
 => => unpacking to docker.io/library/devops-calc:latest                            1.0s
~\..\SPE Calc  main
```

Step 5: Continuous Delivery (Docker Hub)

Brief: Once the CI pipeline verifies the code is good, the application must be packaged and stored in an artifact registry. I used **Docker Hub**.

Setup & Explanation: I expanded my GitHub Actions pipeline to include a CD job. For security, I generated a Personal Access Token on Docker Hub and stored it securely in GitHub Repository Secrets as `DOCKER_USERNAME` and `DOCKER_PASSWORD`. The pipeline automatically signs in, builds the Docker image, and pushes it to the public registry.

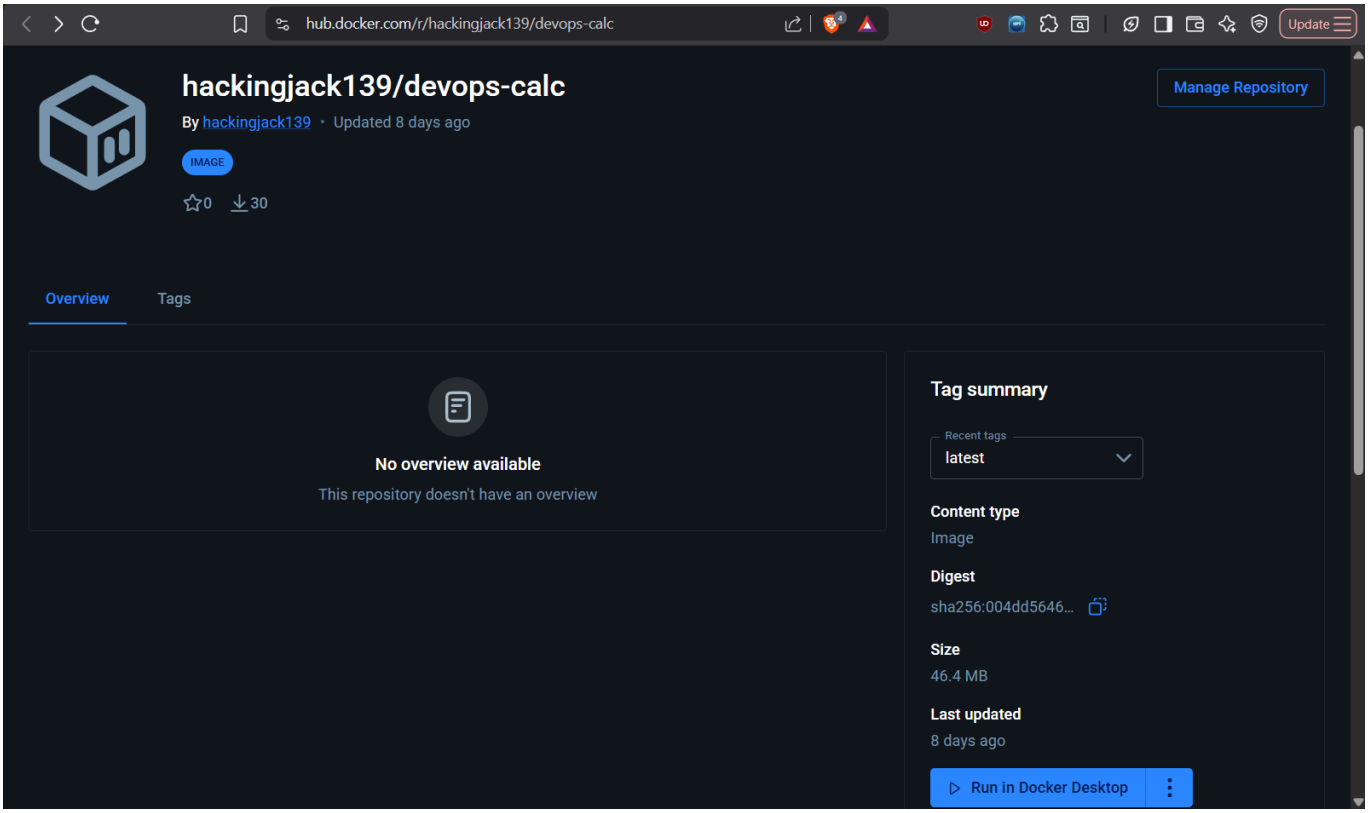
Script (`.github/workflows/ci.yml`):

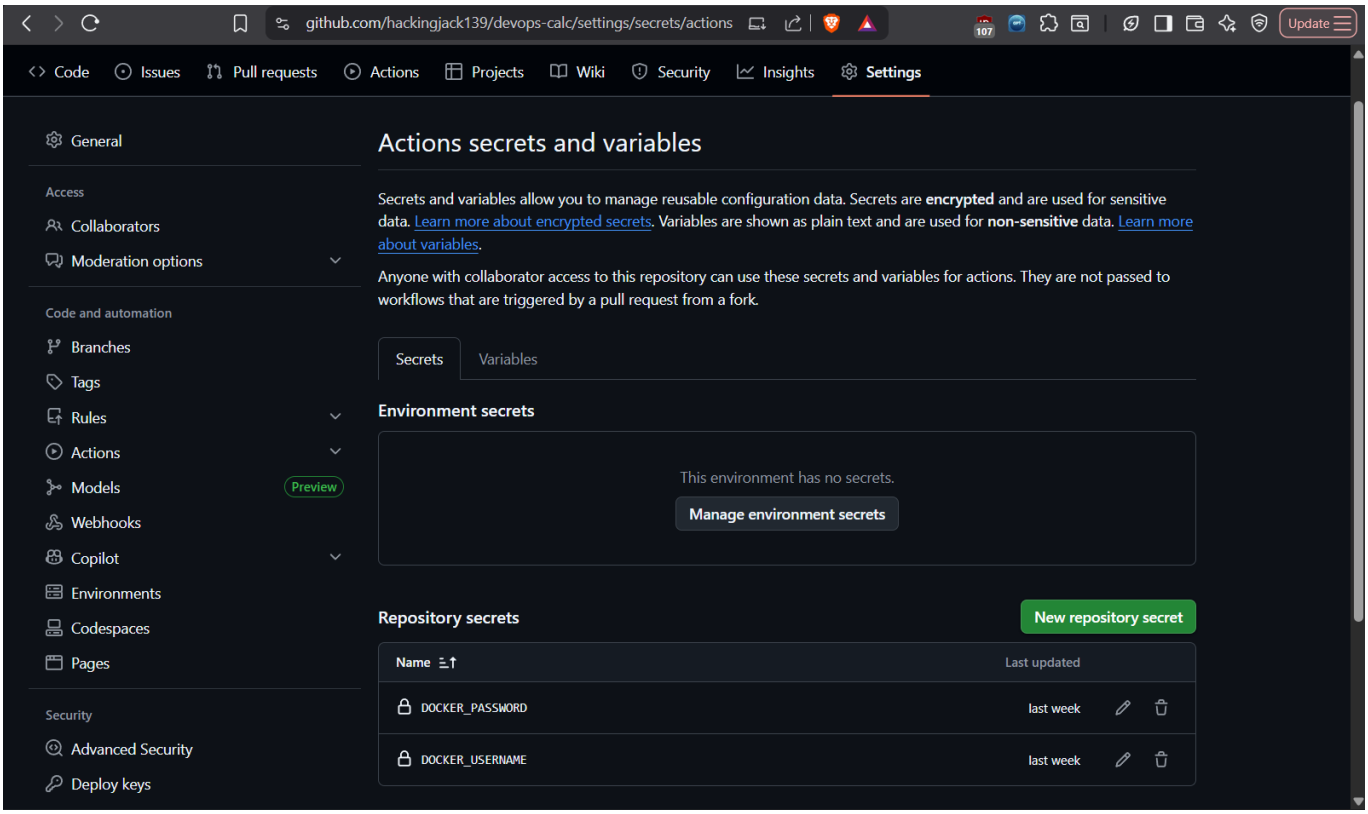
```yaml
name: CI/CD Pipeline
on:
  push:
    branches: [ "main" ]

jobs:
  build_and_test:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - name: Set up Python 3.10
      uses: actions/setup-python@v5
      with:
        python-version: "3.10"
    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r scientific-calc/requirements.txt
    - name: Test with pytest
      working-directory: ./scientific-calc
      run: |
        pytest test_app.py -v

  docker_build_and_push:
    needs: build_and_test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    steps:
      - name: Check out the repo
        uses: actions/checkout@v4
      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}
      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: hackingjack139/devops-calc:latest

  notify:
    needs: [build_and_test, docker_build_and_push]
    if: always()
    runs-on: ubuntu-latest
    steps:
      - name: Send Email Notification
        uses: dawidd6/action-send-mail@v3
        with:
          server_address: smtp.gmail.com
          server_port: 465
          secure: true
```

```
          username: ${{ secrets.EMAIL_USERNAME }}
          password: ${{ secrets.EMAIL_PASSWORD }}
          subject: GitHub Actions job result - ${{ job.status }}
          to: ${{ secrets.DEVELOPER_EMAIL }}
          from: DevOps Pipeline
          body: Pipeline run for ${{ github.repository }} has finished with status
${{ job.status }}.
```

Links:

- Docker Hub Repository: `https://hub.docker.com/r/hackingjack139/devops-calc`





Step 6: Configuration Management & Deployment (Rundeck)

Brief: Configuration Management tools securely orchestrate the deployment of applications to target hosts, ensuring infrastructure is in the correct state. I used **Rundeck**, running inside a local Docker container, to manage the deployment.

Setup & Explanation: Since tools like Ansible and Rundeck do not run natively on Windows, I deployed a local Rundeck server using a custom Docker Compose setup. I created a Rundeck Job definition (`rundeck-job.yml`) that executes the deployment steps:

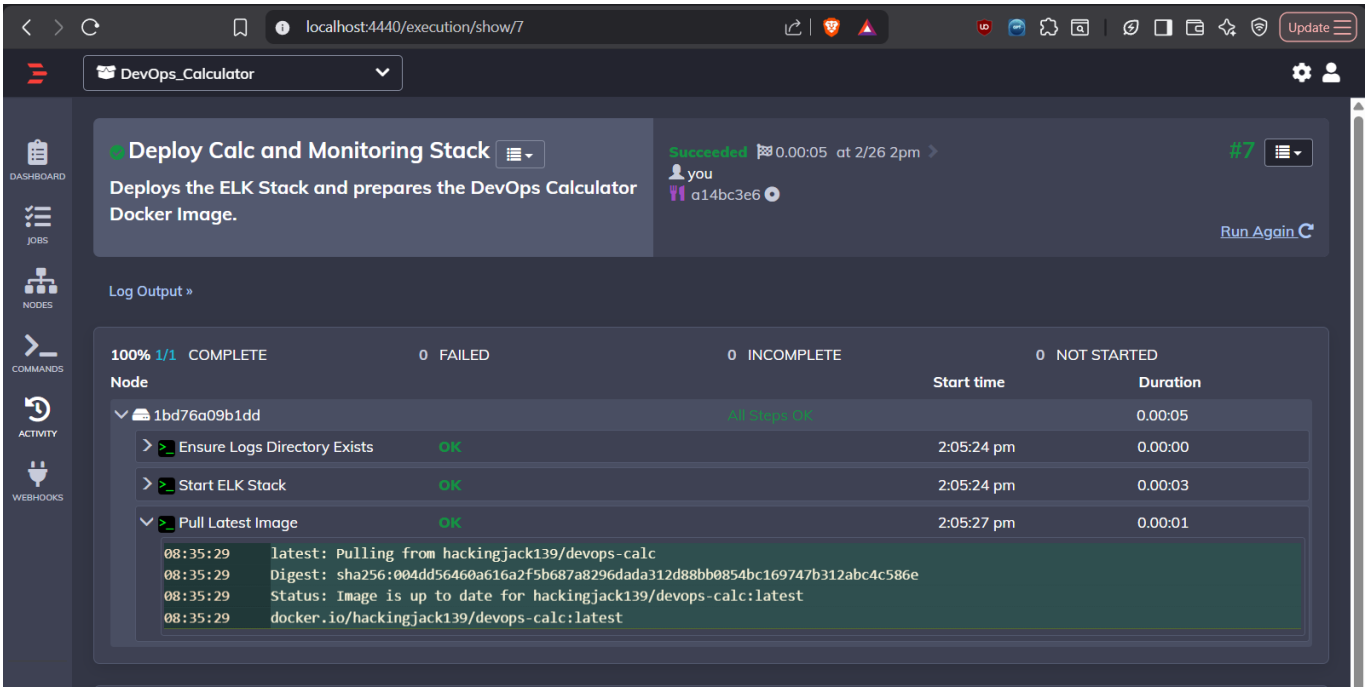
1. Ensure the local logging directory exists.
2. Start the ELK monitoring stack in the background.
3. Pull the latest tested calculator image from Docker Hub.

I then logged into the Rundeck UI at `http://localhost:4440`, uploaded the Job definition, and executed it to instantly deploy my infrastructure.

Script (`deploy/rundeck-job.yml`):

```
- defaultTab: nodes
  description: 'Deploys the ELK Stack and prepares the DevOps Calculator Docker
Image.'
  executionEnabled: true
  id: deploy-devops-calc
  loglevel: INFO
  name: Deploy Calc and Monitoring Stack
  nodeFilterEditable: false
  plugins:
    ExecutionLifecycle: null
```

```
  scheduleEnabled: true
  sequence:
    commands:
    - description: 'Ensure Logs Directory Exists'
      exec: mkdir -p /home/rundeck/project/logs
    - description: 'Start ELK Stack'
      exec: docker compose -f /home/rundeck/project/docker-compose.yml up -d
    - description: 'Pull Latest Image'
      exec: docker pull hackingjack139/devops-calc:latest
    keepgoing: false
    strategy: node-first
  uuid: deploy-devops-calc
```



Step 7: Application Execution

Brief: The application is an interactive CLI calculator. Once Rundeck pulled the image and started the logging infrastructure, the calculator was executed manually attached to my logging volumes.

Setup & Explanation: The application was run via Docker using attached volumes so that the JSON logs it generates inside the container are instantly saved to the shared Docker `applogs` volume that the monitoring stack is watching.

Commands Used:

```
docker run -it --rm -v applogs:/app/logs hackingjack139/devops-calc:latest
```

```
~\..\SPE Calc  main  docker run -it --rm -v applogs:/app/logs hackingjack139/dev
ops-calc:latest
>
Select an option:
1: Square root function - √x
2: Factorial function - x!
3: Natural logarithm (base e) - ln(x)
4: Power function - x^b
5: Exit
2
Enter a value for x:4
Result: 24
Select an option:
1: Square root function - √x
2: Factorial function - x!
3: Natural logarithm (base e) - ln(x)
4: Power function - x^b
5: Exit
t
Invalid input. Please try again.
Select an option:
1: Square root function - √x
2: Factorial function - x!
3: Natural logarithm (base e) - ln(x)
4: Power function - x^b
5: Exit
5
Exiting.
```

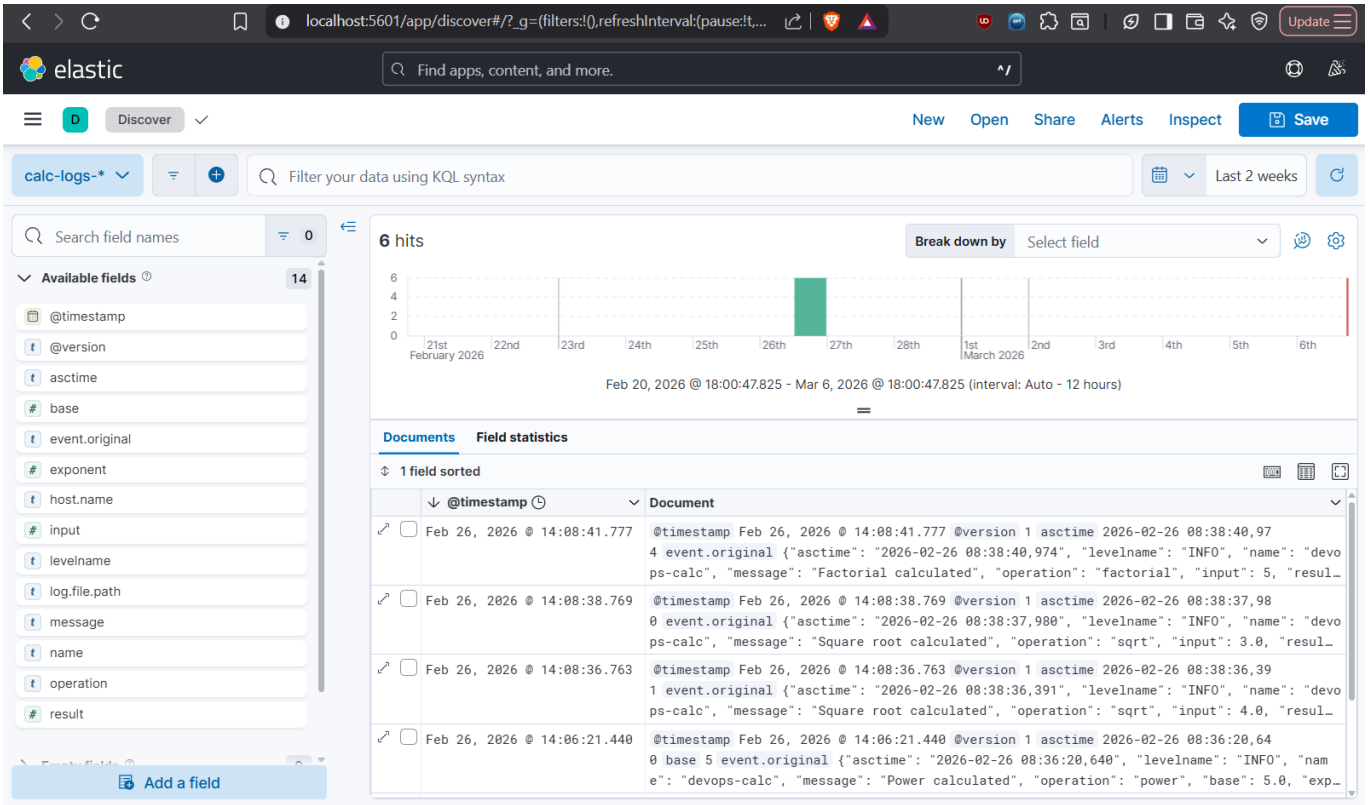
Step 8: Monitoring & Observability (ELK Stack)

Brief: Monitoring provides vital insights into application health and usage. I implemented the **ELK Stack (Elasticsearch, Logstash, Kibana)** dynamically linked to my python calculator.

Setup & Explanation:

1. **Python JSON Logging:** I imported `python-json-logger` into `app.py`. Every calculation, input, and error is now written to `/logs/calc.log` in a machine-readable JSON format.
2. **Logstash Integration:** A custom Logstash container (`Dockerfile_logstash`) was created to ingest `calc.log` from the `applogs` Docker volume and forward it to Elasticsearch.
3. **Kibana Visualization:** I accessed Kibana on port 5601, created an Index Pattern (`calc-logs-*`) to track the Logstash data stream, and successfully viewed the calculator's operations in real-time.

Script (`docker-compose.yml` snippet for ELK):

```yaml
version: '3.7'
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.10.4
    environment:
      - discovery.type=single-node
```

```yaml
      - xpack.security.enabled=false
    ports:
      - "9200:9200"
  logstash:
    build:
      context: .
      dockerfile: Dockerfile_logstash
    volumes:
      - applogs:/usr/share/logstash/logs:ro
  kibana:
    image: docker.elastic.co/kibana/kibana:8.10.4
    ports:
      - "5601:5601"
volumes:
  applogs:
    external: true
```